

Partie java :

1- Classe principale :

- Classe admin :

```
- package com.example.testing01;

public class Admin {
    private int    idAdm;
    private String nom;
    private String prenom;
    private String email;
    private String motDePasse;
    private String service;

    public Admin() {}

    public Admin(String nom, String prenom, String email,
                  String motDePasse, String service) {
        this.nom      = nom;
        this.prenom    = prenom;
        this.email     = email;
        this.motDePasse = motDePasse;
        this.service   = service;
    }

    public int    getIdAdm()      { return idAdm; }
    public void    setIdAdm(int id) { this.idAdm = id; }

    public String  getNom()        { return nom; }
    public void    setNom(String n) { this.nom = n; }

    public String  getPrenom()     { return prenom; }
    public void    setPrenom(String p) { this.prenom = p; }

    public String  getEmail()      { return email; }
    public void    setEmail(String e) { this.email = e; }

    public String  getMotDePasse() { return motDePasse; }
    public void    setMotDePasse(String mp) { this.motDePasse = mp; }

    public String  getService()    { return service; }
    public void    setService(String s) { this.service = s; }

    @Override
    public String toString() {
        return prenom + " " + nom + " (" + service + ")";
    }
}
```

- classe etudiant :

```
- package com.example.testing01;
import java.time.LocalDate;
public class Etudiant {

    private int        idEtd;
    private String      nom;
    private String      prenom;
    private String      adresse;
    private String      motDePasse;
    private String      telephone;
    private String      email;
    private String      filier;
    private LocalDate   dateNaissance;
    private String      sexe;        // "M" ou "F"
    private String      statut;      // "actif" ou "inactif"

    public Etudiant() {}
    public Etudiant(String nom, String prenom, String email,
                    String telephone, String motDePasse) {
        this.nom        = nom;
        this.prenom      = prenom;
        this.email       = email;
        this.telephone   = telephone;
        this.motDePasse  = motDePasse;
        this.statut      = "inactif";
    }

    public int        getIdEtd()        { return idEtd; }
    public void        setIdEtd(int id)  { this.idEtd = id; }

    public String      getNom()          { return nom; }
    public void        setNom(String n)  { this.nom = n; }

    public String      getPrenom()       { return prenom; }
    public void        setPrenom(String p){ this.prenom = p; }

    public String      getAdresse()      { return adresse; }
    public void        setAdresse(String a){ this.adresse = a; }

    public String      getMotDePasse()   { return motDePasse; }
    public void        setMotDePasse(String mp){ this.motDePasse
= mp; }

    public String      getTelephone()    { return telephone; }
    public void        setTelephone(String t){ this.telephone =
t; }

    public String      getEmail()        { return email; }
    public void        setEmail(String e) { this.email = e; }

    public String      getFilier()       { return filier; }
    public void        setFilier(String f){ this.filier = f; }

    public LocalDate   getDateNaissance() { return dateNaissance; }
    public void        setDateNaissance(LocalDate d){
this.dateNaissance = d; }

    public String      getSexe()         { return sexe; }
    public void        setSexe(String s) { this.sexe = s; }
```

```

        public String    getStatut()          { return statut; }
        public void      setStatut(String s){ this.statut = s; }

        @Override
        public String toString() {
            return prenom + " " + nom + " (" + email + ")";
        }
    }

```

- reservation :

```

- //classe Reservation
package com.example.testing01;
import java.time.LocalDateTime;
public class Reservation {

    private int          idRes;
    private LocalDateTime dateHrRes;
    private int          nbPlaces;
    private String       statut;
    private String       moyenPaie;
    private double       montantTotal;
    private int          idEtudiant;
    private int          idTrajet;
    public Reservation() {
        this.statut      = "en_attente";
        this.moyenPaie   = "especes";
        this.nbPlaces    = 1;
    }
    public Reservation(int idEtudiant, int idTrajet,
                       int nbPlaces, String moyenPaie, double
montant) {
        this.idEtudiant  = idEtudiant;
        this.idTrajet    = idTrajet;
        this.nbPlaces    = nbPlaces;
        this.moyenPaie   = moyenPaie;
        this.montantTotal = montant;
        this.dateHrRes   = LocalDateTime.now();
        this.statut      = "en_attente";
    }
    public int          getIdRes()            { return idRes;
}
    public void         setIdRes(int id)      { this.idRes =
id; }

    public LocalDateTime getDateHrRes()       { return
dateHrRes; }
    public void         setDateHrRes(LocalDateTime d){
this.dateHrRes = d; }

    public int          getNbPlaces()         { return
nbPlaces; }
    public void         setNbPlaces(int n)    { this.nbPlaces
= n; }

    public String       getStatut()           { return
statut; }
    public void         setStatut(String s)   { this.statut =
s; }

```

```

        public String      getMoyenPaie()      { return
moyenPaie; }
        public void      setMoyenPaie(String m){
this.moyenPaie = m; }

        public double      getMontantTotal()    { return
montantTotal; }
        public void      setMontantTotal(double m){
this.montantTotal = m; }

        public int      getIdEtudiant()      { return
idEtudiant; }
        public void      setIdEtudiant(int id) {
this.idEtudiant = id; }

        public int      getIdTrajet()          { return
idTrajet; }
        public void      setIdTrajet(int id)   { this.idTrajet
= id; }

        @Override
        public String toString() {
            return "Réservation #" + idRes + " - " + nbPlaces + "
place(s) - " + statut;
        }
    }
}

```

- Classe trajet :

```

- package com.example.testing01;

import java.sql.Time;

public class Trajet {
    private int idTraj;
    private String lieuDepart;
    private String destination;
    private double distanceKm;
    private Time dureeEstimee;
    private boolean disponi;

    // Constructeurs
    public Trajet() {}

    public Trajet(int idTraj, String lieuDepart, String
destination, double distanceKm, Time dureeEstimee, boolean
disponi) {
        this.idTraj = idTraj;
        this.lieuDepart = lieuDepart;
        this.destination = destination;
        this.distanceKm = distanceKm;
        this.dureeEstimee = dureeEstimee;
        this.disponi = disponi;
    }

    // Getters & Setters
    public int getIdTraj() { return idTraj; }
    public void setIdTraj(int idTraj) { this.idTraj = idTraj; }

    public String getLieuDepart() { return lieuDepart; }
}

```

```

        public void setLieuDepart(String lieuDepart) {
            this.lieuDepart = lieuDepart; }

        public String getDestination() { return destination; }
        public void setDestination(String destination) {
            this.destination = destination; }

        public double getDistanceKm() { return distanceKm; }
        public void setDistanceKm(double distanceKm) {
            this.distanceKm = distanceKm; }

        public Time getDureeEstimee() { return dureeEstimee; }
        public void setDureeEstimee(Time dureeEstimee) {
            this.dureeEstimee = dureeEstimee; }

        public boolean isDisponi() { return disponi; }
        public void setDisponi(boolean disponi) { this.disponi =
            disponi; }
    }

```

- les classe controller :

admincontroller :

```

- package com.example.testing01;

import javafx.beans.property.*;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.stage.Stage;
import java.io.IOException;
import java.sql.*;

public class AdminController {

    // — En-tête —
    @FXML private Label lblAdminInfo;

    // — Onglet Trajets —
    @FXML private TextField tfLieuDep, tfDest, tfDistance,
    tfDuree;
    @FXML private TableView<Trajet> tableTrajetsAdmin;
    @FXML private TableColumn<Trajet, Integer> colTrajId;
    @FXML private TableColumn<Trajet, String> colTrajDep,
    colTrajDest, colTrajDuree;
    @FXML private TableColumn<Trajet, Double> colTrajDist;
    @FXML private TableColumn<Trajet, Boolean> colTrajDispo;

    // — Onglet Réservations —
    @FXML private TableView<Reservation>
    tableReservationsAdmin;
    @FXML private TableColumn<Reservation, Integer> colResId,

```

```

colResPlaces, colResEtu, colResTrajId;
@FXML private TableColumn<Reservation, String> colResDate,
colResStatut, colResMoyen;
@FXML private TableColumn<Reservation, Double>
colResMontant;

// — Onglet Étudiants —
@FXML private TableView<Etudiant> tableEtudiants;
@FXML private TableColumn<Etudiant, Integer> colEtuId;
@FXML private TableColumn<Etudiant, String> colEtuNom,
colEtuPrenom, colEtuEmail,
colEtuTel, colEtuStatut;
private Admin admin;

@FXML
public void initialize() {
    // Colonnes Trajets
    colTrajId .setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().getIdTraj()).asObject());
    colTrajDep .setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().getLieuDepart()));
    colTrajDest .setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().getDestination()));
    colTrajDist .setCellValueFactory(d -> new
SimpleDoubleProperty(d.getValue().getDistanceKm()).asObject());
    colTrajDuree.setCellValueFactory(d -> new
SimpleStringProperty(
        d.getValue().getDureeEstimee() != null ?
d.getValue().getDureeEstimee().toString() : ""));
    colTrajDispo.setCellValueFactory(d -> new
SimpleBooleanProperty(d.getValue().isDisponi()).asObject());

    // Colonnes Réservations
    colResId .setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().getIdRes()).asObject());
    colResDate .setCellValueFactory(d -> new
SimpleStringProperty(
        d.getValue().getDateHrRes() != null ?
d.getValue().getDateHrRes().toString() : ""));
    colResPlaces .setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().getNbPlaces()).asObject());
    colResStatut .setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().getStatut()));
    colResMoyen .setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().getMoyenPaieM()));
    colResMontant.setCellValueFactory(d -> new
SimpleDoubleProperty(d.getValue().getMontantTotal()).asObject()
);
    colResEtu .setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().getIdEtudiant()).asObject()
);
    colResTrajId .setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().getIdTrajet()).asObject());

    // Colonnes Étudiants
    colEtuId .setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().getIdEtd()).asObject());
    colEtuNom .setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().getNom()));
    colEtuPrenom.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().getPrenom()));

```

```

        colEtuEmail .setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().getEmail()));
        colEtuTel .setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().getTelephone()));
        colEtuStatut.setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().getStatut()));
    }

    // Initialisation après login
    public void setAdmin(Admin admin) {
        this.admin = admin;
        lblAdminInfo.setText("Connecté : " + admin.getPrenom()
+ " "
+ admin.getNom() + " (" + admin.getService() +
")");
        rafraichirTrajets();
        rafraichirReservations();
        rafraichirEtudiants();
    }

    //Gestion des Trajets
    @FXML
    public void ajouterTrajet() {
        String lieuDep = tfLieuDep.getText().trim();
        String dest = tfDest.getText().trim();

        if (lieuDep.isEmpty() || dest.isEmpty()
|| tfDistance.getText().isBlank() ||
tfDuree.getText().isBlank()) {
            showAlert("Remplissez tous les champs.");
            return;
        }

        try {
            double distance =
Double.parseDouble(tfDistance.getText().trim());
            Time duree =
Time.valueOf(tfDuree.getText().trim());

            String sql = "INSERT INTO trajet (lieu_dpr,
destination, distance_km, duree_estimee, disponi) "
+ "VALUES (?, ?, ?, ?, TRUE)";

            try (Connection conn = connection.getConnection();
PreparedStatement stmt =
conn.prepareStatement(sql)) {
                stmt.setString(1, lieuDep);
                stmt.setString(2, dest);
                stmt.setDouble(3, distance);
                stmt.setTime(4, duree);
                stmt.executeUpdate();
            }
            tfLieuDep.clear(); tfDest.clear();
            tfDistance.clear(); tfDuree.clear();
            rafraichirTrajets();
            showAlert("Trajet ajouté !");

        } catch (NumberFormatException e) {
            showAlert("Distance invalide .");
        } catch (IllegalArgumentException e) {
            showAlert("Format de durée invalide .");
        }
    }

```

```

        } catch (SQLException e) {
            showAlert("Erreur : " + e.getMessage());
        }
    }

    @FXML
    public void supprimerTrajet() {
        Trajet t =
tableTrajetsAdmin.getSelectionModel().getSelectedItem();
        if (t == null) { showAlert("Sélectionnez un trajet.");
return; }

        try (Connection conn = connection.getConnection();
            PreparedStatement stmt = conn.prepareStatement(
                "DELETE FROM trajet WHERE id_traj = ?")) {
            stmt.setInt(1, t.getIdTraj());
            stmt.executeUpdate();
            rafraichirTrajets();
        } catch (SQLException e) {
            showAlert("Erreur : " + e.getMessage());
        }
    }

    //Gestion des Réservations
    @FXML public void validerReservation() {
changerStatut("confirmee"); }
    @FXML public void refuserReservation() {
changerStatut("annulee"); }

    private void changerStatut(String statut) {
        Reservation r =
tableReservationsAdmin.getSelectionModel().getSelectedItem();
        if (r == null) { showAlert("Sélectionnez une
réservation."); return; }

        try (Connection conn = connection.getConnection();
            PreparedStatement stmt = conn.prepareStatement(
                "UPDATE reservation SET statut = ? WHERE
id_res = ?")) {
            stmt.setString(1, statut);
            stmt.setInt(2, r.getIdRes());
            stmt.executeUpdate();
            rafraichirReservations();
        } catch (SQLException e) {
            showAlert("Erreur : " + e.getMessage());
        }
    }

    // Déconnexion
    @FXML
    public void seDeconnecter() {
        try {
            Parent root = FXMLLoader.load(
getClass().getResource("/com/example/testing01/hello-
view.fxml"));
            Stage stage = (Stage)
lblAdminInfo.getScene().getWindow();
            stage.setScene(new Scene(root));
            stage.centerOnScreen();
        } catch (IOException e) {

```

```

        showAlert("Erreur de navigation : " +
e.getMessage());
    }

    //Chargement des données
    private void rafraichirTrajets() {
        ObservableList<Trajet> liste =
FXCollections.observableArrayList();
        try (Connection conn = connection.getConnection();
            ResultSet rs =
conn.createStatement().executeQuery("SELECT * FROM trajet")) {
            while (rs.next()) {
                Trajet t = new Trajet();
                t.setIdTraj(rs.getInt("id_traj"));
                t.setLieuDepart(rs.getString("lieu_dpr"));
                t.setDestination(rs.getString("destination"));
                t.setDistanceKm(rs.getDouble("distance_km"));
                t.setDureeEstimee(rs.getTime("duree_estimee"));
                t.setDisponi(rs.getBoolean("disponi"));
                liste.add(t);
            }
        } catch (SQLException e) { e.printStackTrace(); }
        tableTrajetsAdmin.setItems(liste);
    }

    private void rafraichirReservations() {
        ObservableList<Reservation> liste =
FXCollections.observableArrayList();
        try (Connection conn = connection.getConnection();
            ResultSet rs =
conn.createStatement().executeQuery(
                "SELECT * FROM reservation WHERE statut =
'en_attente')) {
            while (rs.next()) {
                Reservation r = new Reservation();
                r.setIdRes(rs.getInt("id_res"));

                r.setDateHrRes(rs.getTimestamp("date_hr_res").toLocalDateTime());
            };

            r.setNbPlaces(rs.getInt("nb_places"));
            r.setStatut(rs.getString("statut"));
            r.setMoyenPaie(rs.getString("moyen_paie"));

            r.setMontantTotal(rs.getDouble("montant_total"));
            r.setIdEtudiant(rs.getInt("id_etd"));
            r.setIdTrajet(rs.getInt("id_traj"));
            liste.add(r);
        }
    } catch (SQLException e) { e.printStackTrace(); }
    tableReservationsAdmin.setItems(liste);
}

    private void rafraichirEtudiants() {
        ObservableList<Etudiant> liste =
FXCollections.observableArrayList();
        try (Connection conn = connection.getConnection();
            ResultSet rs =
conn.createStatement().executeQuery("SELECT * FROM etudiant"))
        {
            while (rs.next()) {

```

```

        Etudiant e = new Etudiant();
        e.setIdEtd(rs.getInt("id_etd"));
        e.setNom(rs.getString("nom_etd"));
        e.setPrenom(rs.getString("prenom_etd"));
        e.setEmail(rs.getString("email_etd"));
        e.setTelephone(rs.getString("telephone_etd"));
        e.setStatut(rs.getString("statut"));
        liste.add(e);
    }
} catch (SQLException e) { e.printStackTrace(); }
tableEtudiants.setItems(liste);
}

private void showAlert(String msg) {
    Alert alert = new Alert(Alert.AlertType.WARNING);
    alert.setContentText(msg);
    alert.showAndWait();
}
}

```

- etudiantcontroller :

```

- package com.example.testing01;

import javafx.beans.property.*;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.stage.Stage;
import java.io.IOException;
import java.sql.*;
import java.time.LocalDateTime;

public class EtudiantController {

    @FXML private Label welcomeLabel;

    // Onglet Trajets
    @FXML private RadioButton rbAller, rbRetour;
    @FXML private ComboBox<String> cmbZone, cmbPaiement;
    @FXML private TableView<Trajet> tableTrajets;
    @FXML private TableColumn<Trajet, Integer> colTrajId;
    @FXML private TableColumn<Trajet, String> colTrajDep,
colTrajDest, colTrajDuree;
    @FXML private TableColumn<Trajet, Double> colTrajDist;
    @FXML private TableColumn<Trajet, Boolean> colTrajDispo;
    @FXML private Spinner<Integer> spinnerPlaces;

    //onglet Mes réservations
    @FXML private TableView<Reservation> tableMesReservations;
    @FXML private TableColumn<Reservation, Integer> colResId,
colResPlaces, colResTraj;
    @FXML private TableColumn<Reservation, String> colResDate,
colResStatut, colResMoyen;
    @FXML private TableColumn<Reservation, Double>
colResMontant;
}

```

```

        // Onglet Profil
        @FXML private TextField tfProfilNom, tfProfilPrenom,
        tfProfilEmail,
            tfProfilTel, tfProfilAdresse, tfProfilFilier;
        @FXML private Label lblProfilStatut;

        private Etudiant etudiant;

        private static final String EHEI = "EHEI Oujda";
        private static final String[] ZONES = {
            "Hay Andalous", "Hay Al Qods", "Hay Salam", "Centre
Ville",
            "Hay Lazaret", "Hay Nassim", "Hay El Wafa", "Gare
Routière"
        };

        @FXML
        public void initialize() {
            cmbPaielement.getItems().addAll("especes", "carte",
"virement");
            cmbPaielement.setValue("especes");
            cmbZone.getItems().addAll(ZONES);

            ToggleGroup tg = new ToggleGroup();
            rbAller.setToggleGroup(tg);
            rbRetour.setToggleGroup(tg);
            rbAller.setSelected(true);

            // Colonnes Trajets
            colTrajId .setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().getIdTraj()).asObject());
            colTrajDep .setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().getLieuDepart()));
            colTrajDest .setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().getDestination()));
            colTrajDist .setCellValueFactory(d -> new
SimpleDoubleProperty(d.getValue().getDistanceKm()).asObject());
            colTrajDuree .setCellValueFactory(d -> new
SimpleStringProperty(
                d.getValue().getDureeEstimee() != null ?
d.getValue().getDureeEstimee().toString() : ""));
            colTrajDispo .setCellValueFactory(d -> new
SimpleBooleanProperty(d.getValue().isDisponible()).asObject());

            // Colonnes Réservations
            colResId .setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().getIdRes()).asObject());
            colResDate .setCellValueFactory(d -> new
SimpleStringProperty(
                d.getValue().getDateHrRes() != null ?
d.getValue().getDateHrRes().toString() : ""));
            colResPlaces .setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().getNbPlaces()).asObject());
            colResStatut .setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().getStatut()));
            colResMoyen .setCellValueFactory(d -> new
SimpleStringProperty(d.getValue().getMoyenPaie()));
            colResMontant .setCellValueFactory(d -> new
SimpleDoubleProperty(d.getValue().getMontantTotal()).asObject()
);

```

```

        colResTraj        .setCellValueFactory(d -> new
SimpleIntegerProperty(d.getValue().getIdTrajet()).asObject());
    }

    // Initialisation après login
    public void setEtudiant(Etudiant e) {
        this.etudiant = e;
        welcomeLabel.setText("Bienvenue, " + e.getPrenom() + "
" + e.getNom());
        tfProfilNom        .setText(e.getNom());
        tfProfilPrenom     .setText(e.getPrenom());
        tfProfilEmail      .setText(e.getEmail());
        tfProfilTel        .setText(e.getTelephone());
        tfProfilAdresse    .setText(e.getAdresse() != null ?
e.getAdresse() : "");
        tfProfilFilier     .setText(e.getFilier()   != null ?
e.getFilier()   : "");
        lblProfilStatut    .setText(e.getStatut());
        chargerMesReservations();
    }

    //Trajets
    @FXML
    public void changerTypeTrajet() {
        tableTrajets.getItems().clear();
        cmbZone.setValue(null);
    }

    @FXML
    public void rechercherTrajets() {
        String zone = cmbZone.getValue();
        if (zone == null || zone.isEmpty()) {
            showAlert("Veuillez sélectionner votre zone.");
            return;
        }

        String sql = "SELECT * FROM trajet WHERE lieu_dpr = ?
AND destination = ? AND disponi = TRUE";
        ObservableList<Trajet> liste =
FXCollections.observableArrayList();

        try (Connection conn = connection.getConnection();
            PreparedStatement stmt =
conn.prepareStatement(sql)) {

            boolean estAller = rbAller.isSelected();
            stmt.setString(1, estAller ? zone : EHEI);
            stmt.setString(2, estAller ? EHEI : zone);

            try (ResultSet rs = stmt.executeQuery()) {
                while (rs.next()) liste.add(buildTrajet(rs));
            }
            tableTrajets.setItems(liste);
            if (liste.isEmpty()) showAlert("Aucun trajet
disponible pour cette sélection.");

        } catch (SQLException e) {
            showAlert("Erreur : " + e.getMessage());
        }
    }
}

```

```

//Réservation
@FXML
public void reserverTrajet() {
    Trajet selected =
tableTrajets.getSelectionModel().getSelectedItem();
    if (selected == null) {
        showAlert("Veuillez sélectionner un trajet.");
        return;
    }

    String sql = "INSERT INTO reservation "
        + "(date_hr_res, nb_places, statut,
moyen_paie, montant_total, id_etd, id_traj) "
        + "VALUES (?, ?, 'en_attente', 'especes', 0, ?,
?)" ;

    try (Connection conn = connection.getConnection();
        PreparedStatement stmt =
conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {

        stmt.setTimestamp(1,
Timestamp.valueOf(LocalDate.now()));
        stmt.setInt(2, spinnerPlaces.getValue());
        stmt.setInt(3, etudiant.getIdEtd());
        stmt.setInt(4, selected.getIdTraj());
        stmt.executeUpdate();

        chargerMesReservations();
        showAlert(" Réservation effectuée avec succès !");

    } catch (SQLException e) {
        showAlert("Erreur : " + e.getMessage());
    }
}

//Profil
@FXML
public void sauvegarderProfil() {
    String sql = "UPDATE etudiant SET nom_etd=?,
prenom_etd=?, email_etd=?, "
        + "telephone_etd=?, adresse=?, filier=? WHERE
id_etd=?";

    try (Connection conn = connection.getConnection();
        PreparedStatement stmt =
conn.prepareStatement(sql)) {

        stmt.setString(1, tfProfilNom.getText().trim());
        stmt.setString(2, tfProfilPrenom.getText().trim());
        stmt.setString(3, tfProfilEmail.getText().trim());
        stmt.setString(4, tfProfilTel.getText().trim());
        stmt.setString(5,
tfProfilAdresse.getText().trim());
        stmt.setString(6, tfProfilFilier.getText().trim());
        stmt.setInt(7, etudiant.getIdEtd());
        stmt.executeUpdate();

        // Mise à jour de l'objet en mémoire
        etudiant.setNom(tfProfilNom.getText().trim());

        etudiant.setPrenom(tfProfilPrenom.getText().trim());

```

```

        etudiant.setEmail(tfProfilEmail.getText().trim());

etudiant.setTelephone(tfProfilTel.getText().trim());

etudiant.setAdresse(tfProfilAdresse.getText().trim());

etudiant.setFilier(tfProfilFilier.getText().trim());

        welcomeLabel.setText("Bienvenue, " +
etudiant.getPrenom() + " " + etudiant.getNom());
        showAlert(" Profil sauvegardé !");

    } catch (SQLException e) {
        showAlert("Erreur : " + e.getMessage());
    }
}

//Déconnexion
@FXML
public void seDeconnecter() {
    naviguerVers("/com/example/testing01/hello-view.fxml",
        (Stage) welcomeLabel.getScene().getWindow());
}

// Méthodes privées
private void chargerMesReservations() {
    ObservableList<Reservation> liste =
FXCollections.observableArrayList();
    String sql = "SELECT * FROM reservation WHERE id_etd =
?";

    try (Connection conn = connection.getConnection();
        PreparedStatement stmt =
conn.prepareStatement(sql)) {

        stmt.setInt(1, etudiant.getIdEtd());
        try (ResultSet rs = stmt.executeQuery()) {
            while (rs.next())
liste.add(buildReservation(rs));
        }
        tableMesReservations.setItems(liste);

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

//Construit
private Trajet buildTrajet(ResultSet rs) throws
SQLException {
    Trajet t = new Trajet();
    t.setIdTraj(rs.getInt("id_traj"));
    t.setLieuDepart(rs.getString("lieu_dpr"));
    t.setDestination(rs.getString("destination"));
    t.setDistanceKm(rs.getDouble("distance_km"));
    t.setDureeEstimee(rs.getTime("duree_estimee"));
    t.setDisponi(rs.getBoolean("disponi"));
    return t;
}

```

```

        private Reservation buildReservation(ResultSet rs) throws
SQLException {
            Reservation r = new Reservation();
            r.setIdRes(rs.getInt("id_res"));

            r.setDateHrRes(rs.getTimestamp("date_hr_res").toLocalDateTime()
);
            r.setNbPlaces(rs.getInt("nb_places"));
            r.setStatut(rs.getString("statut"));
            r.setMoyenPaie(rs.getString("moyen_paie"));
            r.setMontantTotal(rs.getDouble("montant_total"));
            r.setIdEtudiant(rs.getInt("id_etd"));
            r.setIdTrajet(rs.getInt("id_traj"));
            return r;
        }

        private void naviguerVers(String fxmlPath, Stage stage) {
            try {
                Parent root =
FXMLLoader.load(getClass().getResource(fxmlPath));
                stage.setScene(new Scene(root));
                stage.centerOnScreen();
            } catch (IOException e) {
                showAlert("Erreur de navigation : " +
e.getMessage());
            }
        }

        private void showAlert(String msg) {
            Alert alert = new Alert(Alert.AlertType.INFORMATION);
            alert.setContentText(msg);
            alert.showAndWait();
        }
    }
}

```

- classe de connexion base de donne

```

- package com.example.testing01;

import java.sql.Connection; //db
import java.sql.DriverManager; //cnx
import java.sql.SQLException;
public class connection {

    private static final String URL =
"jdbc:mysql://localhost:3306/intelligents_transport_10"; //e ob
    private static final String USER = "root";
    private static final String PASS = "200516";

    private static Connection instance;

    private connection() {}

    public static Connection getConnection() throws
SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }
}

```

- interface login

The screenshot shows a web browser window titled "I-TRANSPORT EHEI Oujda". The login form is centered and contains the following elements:

- Header: I-TRANSPORT EHEI Oujda –
- EMAIL: A text input field.
- MOT DE PASSE: A password input field with masked characters (*****).
- RÔLE: A dropdown menu with "Étudiant" selected.
- SE CONNECTER: A button.

- Interface login etudiant

Recherché de trajet

The screenshot shows a web browser window titled "I-TRANSPORT EHEI Oujda". The interface is for a student and includes the following elements:

- Header: Bienvenue, maafa reda (Welcome, my friend) and a red button labeled Déconnexion (Logout).
- Navigation: Tabs for Rechercher un trajet (Search a route), Mes réservations (My reservations), and Mon profil (My profile).
- Type de trajet: Radio buttons for Zone → EHEI (Aller) (selected) and EHEI → Zone (Retour) (Return).
- Votre zone: A dropdown menu labeled "Sélectionnez votre zone" (Select your zone) with a list of zones: Hay Andalous, Hay Al Qods, Hay Salam, Centre Ville, Hay Lazaret, Hay Nassim, Hay El Wafa, and Gare Routière.
- Rechercher: A blue button with a magnifying glass icon.
- Table: A table with columns ID, Distance km, Durée (Duration), and Disponible (Available). The table is currently empty, with the text "Aucun contenu dans la table" (No content in the table) displayed below it.
- Footer: Places: 1 (with a spinner), Paiement: especes (with a dropdown), and a green button labeled Réserver (Reserve).

[illegible]

I-TRANSPORT EHEI Oujda

Bienvenue, maafa reda [Déconnexion](#)

Rechercher un trajet Mes réservations **Mon profil**

Nom :

Prénom :

Email :

Téléphone :

Adresse :

Filière :

Statut : actif

[Sauvegarder](#)

Rechercher un trajet Mes réservations Mon profil

Pour interface admin :

gestion des trajet

I-TRANSPORT EHEI Oujda

Espace Administrateur

Déconnexion

Connecté : Sophie nassim (Gestion des trajets)

Gestion des trajets

Réservations en attente

Liste des étudiants

Lieu de départ

Destination

Distance (km)

Durée (HH:MM:SS)

Ajouter

ID	Départ	Destination	Distance km	Durée	Disponible	
1	Hay Andalous	EHEI Oujda	5.2	00:15:00	true	
2	Hay Al Qods	EHEI Oujda	4.8	00:13:00	true	
3	Hay Salam	EHEI Oujda	6.1	00:18:00	true	
4	Centre Ville	EHEI Oujda	3.5	00:10:00	true	
5	Hay Lazaret	EHEI Oujda	7.0	00:20:00	true	
6	Hay Nassim	EHEI Oujda	5.5	00:16:00	true	
7	Hay El Wafa	EHEI Oujda	4.2	00:12:00	true	
8	Gare Routière	EHEI Oujda	3.8	00:11:00	true	
9	EHEI Oujda	Hay Andalous	5.2	00:15:00	true	
10	EHEI Oujda	Hay Al Qods	4.8	00:13:00	true	
11	EHEI Oujda	Hay Salam	6.1	00:18:00	true	
12	EHEI Oujda	Centre Ville	3.5	00:10:00	true	
13	EHEI Oujda	Hay Lazaret	7.0	00:20:00	true	
14	EHEI Oujda	Hay Nassim	5.5	00:16:00	true	
15	EHEI Oujda	Hay El Wafa	4.2	00:12:00	true	
16	EHEI Oujda	Gare Routière	3.8	00:11:00	true	

Supprimer sélection

resrvation en attent :

Espace Administrateur

Déconnexion

Connecté : Sophie nassim (Gestion des trajets)

Gestion des trajets Réservations en attente Liste des étudiants

ID	Date	Places	Statut	Paielement	Montant	Étudiant ID	Trajet ID	
Aucun contenu dans la table								

✓ Valider

✗ Refuser

Liste des etudiant

[illegible]

Est bouton se deconnecte pour deconnecte .